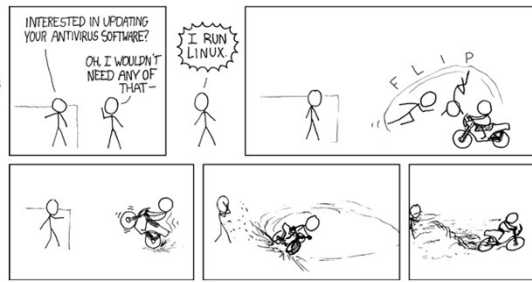




Department of Engineering Cybernetics



TTK4147 Real-time Systems

07 Linux and Real-time Variants

01.10.2025

1

TTK4147 – 2025: Linux and Real-time Variants



1

Today

- Linux
 - Brief History
 - Linux Kernel
 - Linux User Space
- Real-time and Linux
 - Real-time Patch
 - Real-time Microkernel
 - Xenomai
 - Slave Microcontroller
- Embedded Linux
 - How to develop for embedded systems?
 - Host - Target
 - Cross-toolchain
 - Build systems

2

TTK4147 – 2025: Linux and Real-time Variants



2

Linux: Brief History

- UNIX was first released in 1971 by Bell labs.
- MINIX was first released in 1987 by Andrew S. Tanenbaum, a free, open source UNIX-like operating system created for educational purposes.
- Linus Torvalds (a Finnish student of computer science doing his masters) posted, in 1991, on an internet MINIX newsgroup that he was working on a “hobby project” . (Torvalds wanted to buy a UNIX system but didn’t have enough money.)
 - Nothing big and professional.
 - Would never run on anything else than 386, or use other hard drives than the one Linus himself had.
 - Later the same year version 0.01 of Linux was released.
 - Linux standing for ‘Linus’s MINIX’.



3

TTK4147 – 2025: Linux and Real-time Variants



3

Brief History continue

- 2 October 2022 the version 6.0 was released. (Version 6.12 with PREEMT_RT integrated was released November 2024 and version 6.16.9 released 25th Sept. 2025)
- In 2019, > 80% of Linux contributions come from developers paid by big enterprises. Intel tops the list, Microsoft also among the biggest contributors to Linux kernel.
- In 2020, Linux runs on all of the top 500 Supercomputers
- The availability of free, open source software from the GNU project have been the key to the success of Linux (some insist on the name GNU/Linux).



4

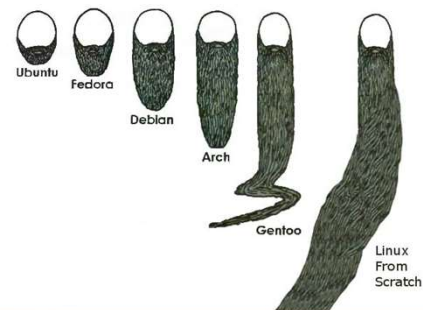
TTK4147 – 2025: Linux and Real-time Variants



4

Linux Distribution

- A Linux system consists of more than just the Linux kernel.
 - Core utilities (often GNU core utils)
 - Libraries
 - Graphic user interface
 - Programs
- Selecting the software a Linux system contains and configure it correctly is NOT trivial.
- If you use a Linux distribution, somebody have done all (or most) of these decisions for you.
- There are currently about 780 Linux distributions on www.distrowatch.com



5

TTK4147 – 2025: Linux and Real-time Variants



5

The Great Linux World Map 2.0 (lots of Linux enthusiasts with spare time)



6

TTK4147 – 2025: Linux and Real-time Variants



6

Linux Kernel

- Full equipped UNIX-like operating system.
- Run on a very large number of processor architecture (even if Torvalds only thought it would run on 386).
- Highly configurable.
- Linux uses the GNU General Public License, making it open source and free to use.
- It is a monolithic kernel, with loadable modules.



7

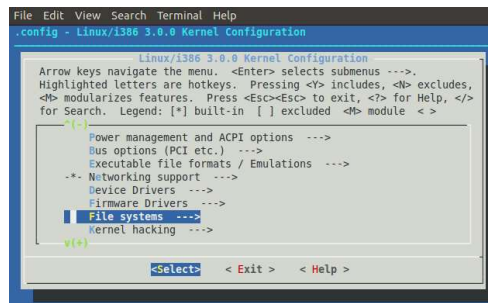
TTK4147 – 2025: Linux and Real-time Variants



7

Building Linux Kernel

- The Linux kernel is a large piece of software (Linux in 2020: 27.8 million lines of code in the kernel) and has countless configuration options.
- Code available on kernel.org.
- The commands
 - “make menuconfig” allows to configure the kernel.
 - “make” builds the kernel.
 - “make modules” builds the kernel modules.



8

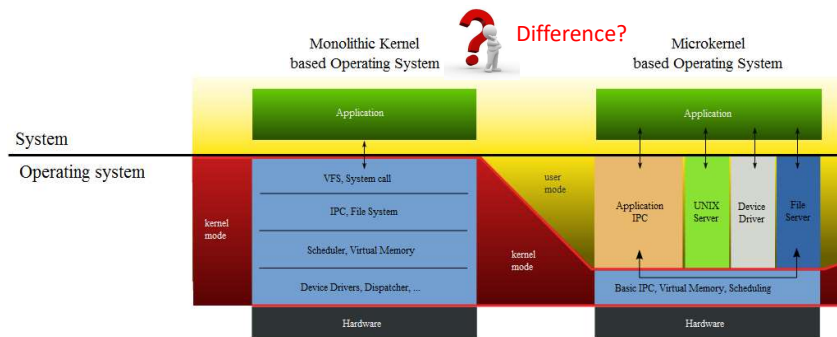
TTK4147 – 2025: Linux and Real-time Variants



8

Modular Monolithic Kernel

- Includes virtually all of the OS functionality in one large block of code that runs as a single process with a single address space
- All the functional components of the kernel have access to all of its internal data structures and routines
- Linux is structured as a collection of modules Difference is what is included in the OS kernel, what must run in kernel mode



9

TTK4147 – 2025: Linux and Real-time Variants



9

Loadable Modules

- Relatively independent blocks
- A module is an object file whose code can be linked to and unlinked from the kernel at run-time
- A module is executed in kernel mode on behalf of the current process
- Have two important characteristics:
 - Dynamic linking
 - Stackable modules
(Hierarchic organized. Individual modules serve as libraries when they are referenced by client modules higher up in the hierarchy, and as clients when they reference modules further down.)

Examples of Linux modules:

- Rfcom – RF communication
- bluetooth
- uio - Userspace I/O framework
- ipv6 - LINUX IPv6 protocol implementation

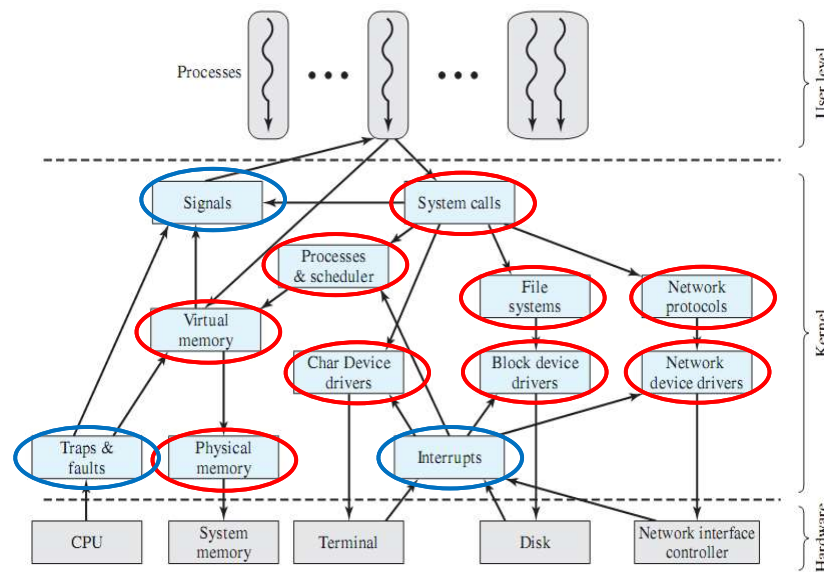
10

TTK4147 – 2025: Linux and Real-time Variants



10

Linux Kernel Components



11

TTK4147 – 2025: Linux and Real-time Variants



11

Concurrency in the Linux Kernel

- Four mechanisms are mentioned in Stallings:
 - Atomic operations**
Operations that are guaranteed to perform without interruption or interference (uniprocessor / multiprocessor).
 - Spinlocks**
Works similarly as a mutex, but threads that want to get the mutex will «spin» until it received it (not sleep, “Busy waiting”).
Effective in situations where the wait time for acquiring a lock is expected to be very short.
 - (Kernel / User level) Semaphores**
Semaphores for kernel-space code are more efficient than user-visible semaphores.
 - Barriers**
Enforce the order in which instructions are executed
(the code should be executed in the order as written in the code, not altered e.g. due to optimizing reasons. Compiler and processor oriented!)

12

TTK4147 – 2025: Linux and Real-time Variants



12

Linux memory management

- Shares many characteristics with UNIX
- Is quite complex

Two main aspects

- process virtual memory
- kernel memory allocation

13

TTK4147 – 2025: Linux and Real-time Variants

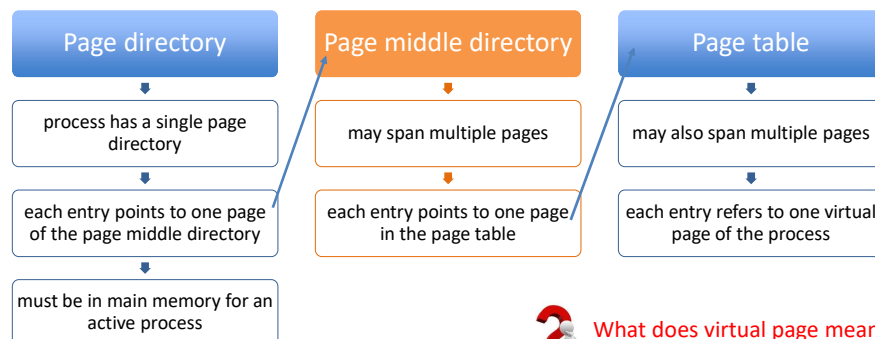
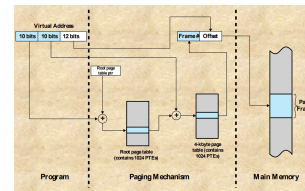
NTNU

13

Linux virtual memory

In virtual memory, the secondary memory (disk) can be accessed as if it was main memory (RAM).

Linux has a three-level page-table structure:



What does virtual page mean?

14

TTK4147 – 2025: Linux and Real-time Variants

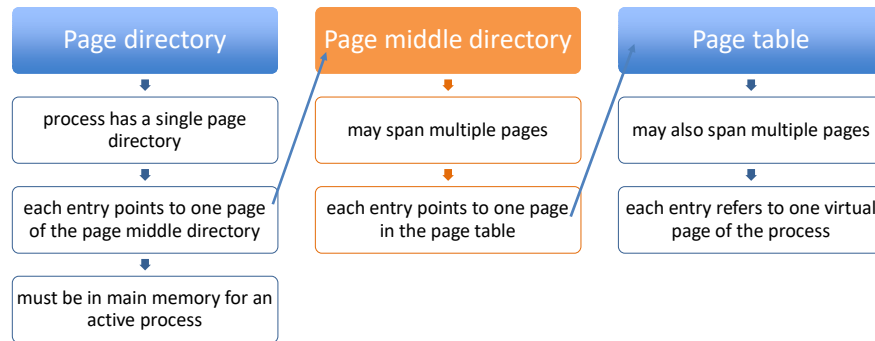
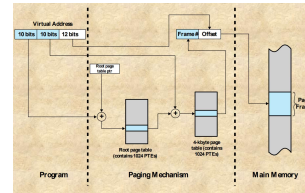
NTNU

14

Linux virtual memory

In virtual memory, the secondary memory (disk) can be accessed as if it was main memory (RAM).

Linux has a three-level page-table structure:



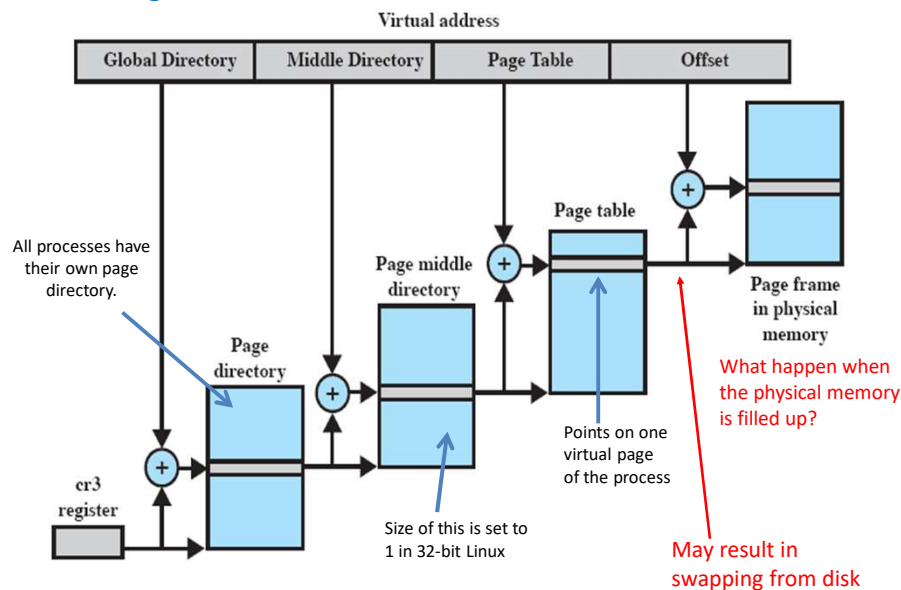
15

TTK4147 – 2025: Linux and Real-time Variants



15

Linux Page Table



16

TTK4147 – 2025: Linux and Real-time Variants

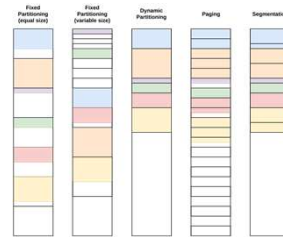


16

Linux Page Replacement

It exists different memory management techniques to allocate memory.

However, the physical memory will sooner or later be filled up



Strategy for replacing pages

- Each page in main memory has a counter, that is incremented each time it is accessed.
- Linux will periodically decrement these counters.
- The lower this counter is, the less frequently has it been used lately.
- A page with a counter of 0 is the best candidate for replacement (Swapped to disk).
- Page replacement in Linux is a form of **Least Frequently Used** policy

17

TTK4147 – 2025: Linux and Real-time Variants



17

Linux Memory Allocation

- **Kernel memory capability** manages physical main memory page frames
 - Primary function is to allocate and deallocate frames for particular uses

Possible owners of a frame include:

- user-space processes
- dynamically allocated kernel data
- static kernel code
- page cache

- A buddy algorithm is used so that memory for the kernel can be allocated and deallocated in units of one or more pages
- Page allocator alone would be inefficient because the kernel requires small short-term memory chunks in odd sizes
- In addition, “Slab allocation”
 - Used by Linux to accommodate small chunks

Request	Size	Allocation	Deallocation
Request 1024	1KB	1024	1024
Request 2048	2KB	2048	2048
Request 4096	4KB	4096	4096
Request 8192	8KB	8192	8192
Request 16384	16KB	16384	16384
Request 32768	32KB	32768	32768
Request 65536	64KB	65536	65536
Request 131072	128KB	131072	131072
Request 262144	256KB	262144	262144
Request 524288	512KB	524288	524288
Request 1048576	1MB	1048576	1048576

18

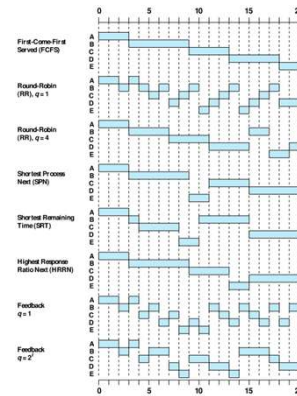
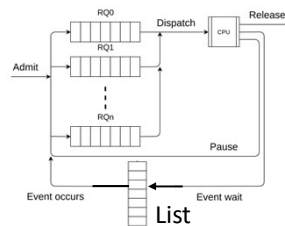
TTK4147 – 2025: Linux and Real-time Variants



18

Linux Scheduling

- An important role of an operating system kernel is to schedule the different processes running on the system.
- The three classes of scheduling are used by Linux:
 - SCHED_FIFO: First-in-first-out **real-time** threads.
 - SCHED_RR: Round-robin **real-time** threads.
 - SCHED_OTHER: Other, all **non-real-time** threads.
- Within each class multiple priorities may be used.
(SCHED_FIFO, SCHED_RR 1 – 99, SCHED_OTHER as default 0)



19

TTK4147 – 2025: Linux and Real-time Variants

NTNU

19

Linux R-T Scheduling, FIFO vs RR

A	minimum
B	middle
C	middle
D	maximum

(a) Relative thread priorities
All are ready when starting,
B arrived before C

D → B → C → A →

(b) Flow with FIFO scheduling



Explain the order
B and C are sliced because equal priority

D → B → C → B → C → A →

(c) Flow with RR scheduling

20

TTK4147 – 2025: Linux and Real-time Variants

NTNU

20

Linux Scheduling

Configure the kernel to be
Fully Preemptible Kernel (Real-Time)

- On paper the three classes of scheduling, each with individual priorities, sounds like what we are looking for in a real-time system, but:
 - The Linux kernel was not intended to be preemptive, and can't be fully preemptive regardless of kernel configuration.
 - Linux desktop distributions are typically running a large number of services and applications (thus the housekeeping takes time!), and it can be difficult to know what can cause a delay in your program.

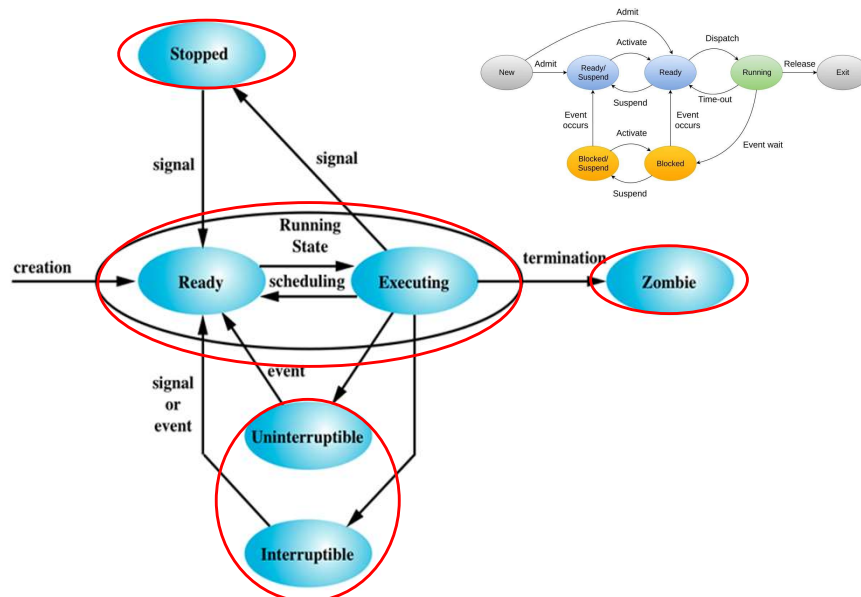
21

TTK4147 – 2025: Linux and Real-time Variants



21

Linux Process / Thread Model



22

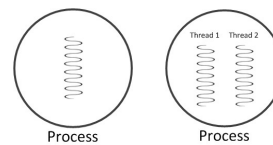
TTK4147 – 2025: Linux and Real-time Variants



Linux Thread Model

EARLIER:

- Originally there were no threads in Linux, only processes.
- Threads could be implemented in user space inside the processes
- One blocked thread would block the whole process



23

TTK4147 – 2025: Linux and Real-time Variants

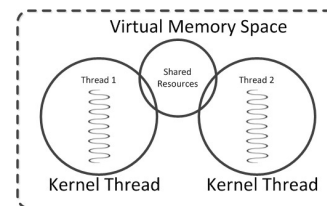


23

Linux Thread Model

NOW:

- Linux does not recognize a distinction between threads and processes
- Processes and threads are *almost* the same, both are running as separate "kernel threads". User-level threads are mapped into kernel-level processes
- Threads share the same virtual memory.
- Context switch between threads are faster than between processes.
- The kernel schedule each kernel thread (both processes and threads) individually.



24

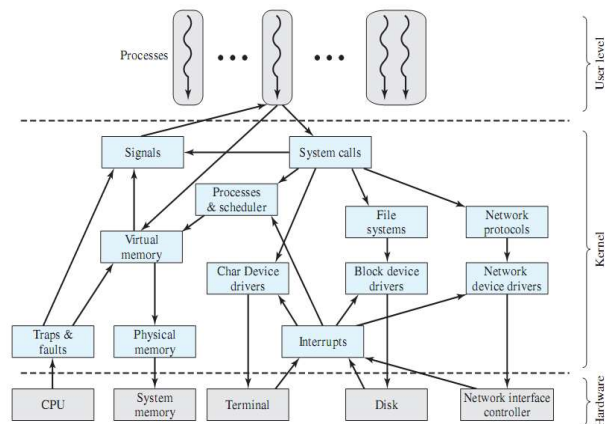
TTK4147 – 2025: Linux and Real-time Variants



24

Linux User Space

- Where «normal» programs run.
- Use user-space library (normally GNU C library, but others can also be used).
- Must use *System Calls* to request kernel services.
- Kernel can notify user-space program with *Signals*.



25

TTK4147 – 2025: Linux and Real-time Variants

NTNU

25

Some Linux Signals

SIGHUP	Terminal hangup	SIGCONT	Continue
SIGQUIT	Keyboard quit	SIGTSTP	Keyboard stop
SIGTRAP	Trace trap	SIGTTOU	Terminal write
SIGBUS	Bus error	SIGXCPU	CPU limit exceeded
SIGKILL	Kill signal	SIGVTALRM	Virtual alarm clock
SIGSEGV	Segmentation violation	SIGWINCH	Window size unchanged
SIGPIPE	Broken pipe	SIGPWR	Power failure
SIGTERM	Termination	SIGRTMIN	First real-time signal
SIGCHLD	Child status unchanged	SIGRTMAX	Last real-time signal

26

TTK4147 – 2025: Linux and Real-time Variants

NTNU

26

Some Linux System Calls

Filesystem related		Interprocess Communication (IPC) related	
close	Close a file descriptor.	msgrcv	A message buffer structure is allocated to receive a message. The system call then reads a message from the message queue specified by msgid into the newly created message buffer.
link	Make a new name for a file.	semctl	Performs the control operation specified by cmd on the semaphore set semid.
open	Open and possibly create a file or device.	semop	Performs operations on selected members of the semaphore set semid.
read	Read from file descriptor.	shmat	Attaches the shared memory segment identified by shmid to the data segment of the calling process.
write	Write to file descriptor.	shmctl	Allows the user to receive information on a shared memory segment, set the owner, group, and permissions of a shared memory segment, or destroy a segment.
Process related		Socket (networking) related	
execve	Execute program.	bind	Assigns the local address and port for a socket. Returns 0 for success and -1 for error.
exit	Terminate the calling process.	connect	Establishes a connection between the given socket and the remote socket associated with sockaddr.
getpid	Get process identification.	gethostname	Returns local host name.
setuid	Set user identity of the current process.	send	Send the bytes contained in buffer pointed to by *msg over the given socket.
ptrace	Provides a means by which a parent process may observe and control the execution of another process, and examine and change its core image and registers.	setsockopt	Sets the options on a socket.
Scheduling related		Miscellaneous	
sched_getparam	Sets the scheduling parameters associated with the scheduling policy for the process identified by pid.	create_module	Attempts to create a loadable module entry and reserve the kernel memory that will be needed to hold the module.
sched_get_priority_max	Returns the maximum priority value that can be used with the scheduling algorithm identified by policy.	fsync	Copies all in-core parts of a file to disk, and waits until the device reports that all parts are on stable storage.
sched_setscheduler	Sets both the scheduling policy (e.g., FIFO) and the associated parameters for the process pid.	query_module	Requests information related to loadable modules from the kernel.
sched_rr_get_interval	Writes into the timespec structure pointed to by the parameter tp the round robin time quantum for the process pid.	time	Returns the time in seconds since January 1, 1970.
sched_yield	A process can relinquish the processor voluntarily without blocking via this system call. The process will then be moved to the end of the queue for its static priority and a new process gets to run.	vhungup	Simulates a hangup on the current terminal. This call arranges for other users to have a "clean" tty at login time.

27

TTK4147 – 2025: Linux and Real-time Variants



27

When User Space is not Enough

Reasons for develop in kernel space:

- User space programs can only access the services already provided by the kernel.
 - You need to access an unsupported device.
 - You need to access a special (unsupported) feature of the CPU of the kernel.
- Functionality can be implemented in kernel space to reduce the number of "context switches" which reduce performance.
 - Example: rotary encoder handling.

Disadvantages with kernel space:

- This probably depends on experience, but most developers will find it more difficult to program in kernel space than in user space.
- You can not use the normal user-space libraries when you program in kernel-space, but instead use Linux kernel header files.

28

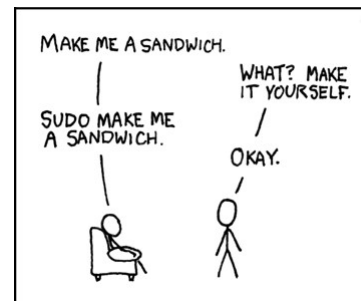
TTK4147 – 2025: Linux and Real-time Variants



28

Root vs Kernel Space

- Root is NOT the same as kernel space.
- Root is the highest level of privilege you can have in user-space.
- Many operations related to kernel-space requires root access, such as:
 - Installing a module.
 - Mounting a hard-drive (although can be configured to auto-mount).
- Root can be a separate user that you must log in as.
- Or you can get root access with the *sudo* (super user do) command.



Root user = Superuser / Admin

Illustrates the power of *sudo*

29

TTK4147 – 2025: Linux and Real-time Variants

NTNU

29

Real-time and Linux

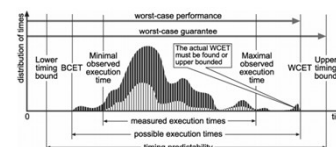
Linux is NOT a real-time system.

- Deterministic “hard real-time” would not be optimal for normal use.
- Kernel is (normally) not preemptive.
- Generally very good performance, but tend to get “outliers” with very high latencies.
- Latency will depend on system load.



Why?

As it will probably have a weak utilization of the CPU



“Soft” Real-time capabilities can be improved.

- Configure the kernel to be “as preemptive” as possible.
- Make sure the system has the smallest possible footprint (by configuration).

30

TTK4147 – 2025: Linux and Real-time Variants

NTNU

30

How to get Hard(er) Real-time in Linux

Three different strategies:

1. Make the Linux kernel fully preemptive, with PREEMPT_RT, thus more predictable.
 - Effect may be uncertain.
2. Add a small real-time co-kernel that runs «below» Linux.
 - RTAI (Real-time application interface)
 - Xenomai (Cobalt core)
3. Outsource the hard real-time tasks to a slave microcontroller
 - A simple microcontroller without OS like AVR provide better control of latencies.
 - Interface between master CPU and slave microcontroller with UART, SPI, I2C etc.

31

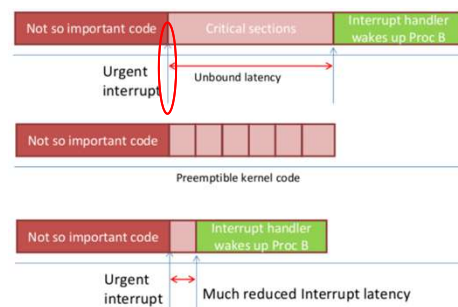
TTK4147 – 2025: Linux and Real-time Variants



31

Strategy 1: PREEMPT-RT

- The default Linux kernel comes in three levels of preemption:
 - No kernel preemption (server)
 - Voluntary kernel preemption (desktop)
 - Preemptive kernel (low-latency desktop)
- Regardless of the level, there are *critical sections* on kernel code that cannot be preempted.
- PREEMPT-RT patch is an effort to reduce the size of such *critical sections*, so that a high priority task can preempt a running task, even kernel task.
- V 6.12 with PREEMT_RT integrated (Nov. 2024), must be activated



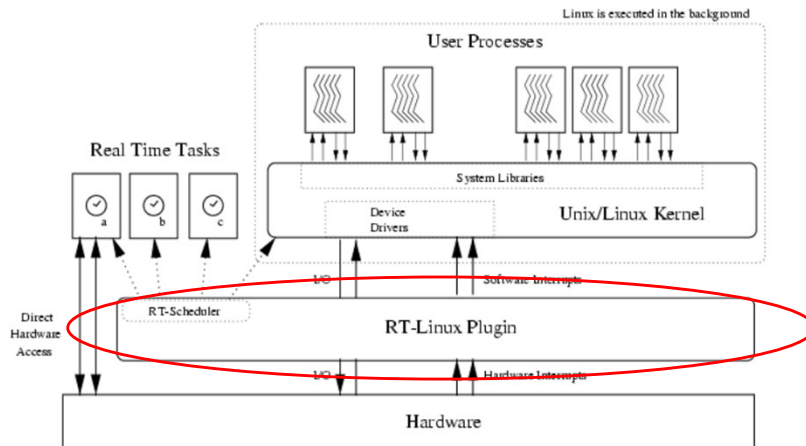
32

TTK4147 – 2025: Linux and Real-time Variants



32

Strategy 2: Generic Linux Co-kernel Concept



33

TTK4147 – 2025: Linux and Real-time Variants

NTNU

33

Xenomai

- Previous to version 3, Xenomai used a co-kernel or dual kernel approach.
- The relatively new xenomai version 3 have two different modes:
 - A dual kernel approach, where the *Cobalt core* runs beside the Linux kernel.
 - A single kernel approach, where the native Linux kernel, normally patched with PREEMPT-RT, forming the *Mercury core*.
- These two modes have the same programming interface (API), thus you can move applications between them.

34

TTK4147 – 2025: Linux and Real-time Variants

NTNU

34

Xenomai - Dual Kernel Approach

- The best performance, lowest latency.
- Challenges with operating in two contexts.
 - One Linux non real-time context.
 - One Xenomai/Cobalt real-time context.
- Switches between them will take time.
 - Also, if you switch to Linux context, you will loose real-time "privileges".
- Additional development will be necessary.
 - Not all libraries, drivers and programs in Linux can run in Xenomai/Cobalt context.
 - Dual kernel patches only work on some specific Linux kernel versions.

35

TTK4147 – 2025: Linux and Real-time Variants



35

Real-time Device Model (RTDM)

- If a Xenomai program use a Linux driver, e.g. some hardware device, the driver will execute in Linux context, not Xenomai.
 - Not hard real-time.
 - Inefficient due to many switches between Linux and Xenomai.
- RTDM can be used to develop full Xenomai drivers.
 - Additional work.
 - Specialized knowledge required.
 - Can be very difficult.
 - Documentation is lacking.

→ Not mature

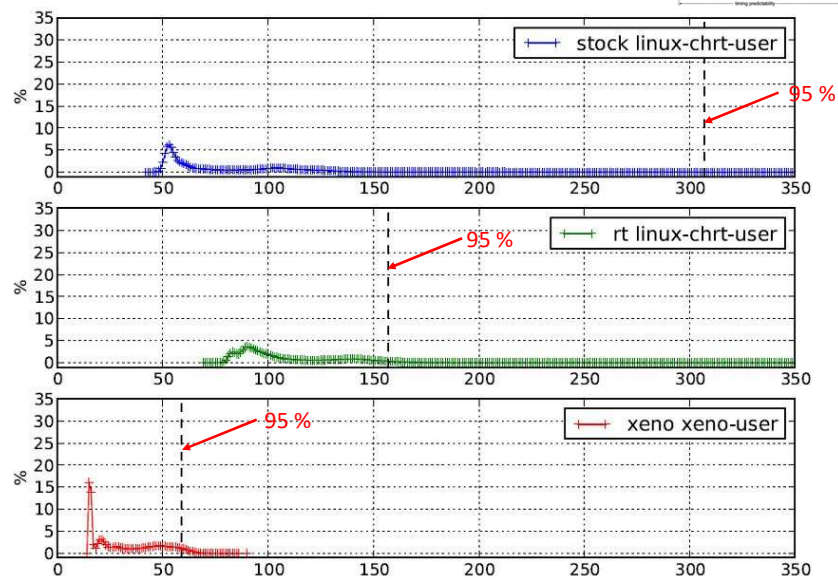
36

TTK4147 – 2025: Linux and Real-time Variants



36

Execution time comparison between solutions



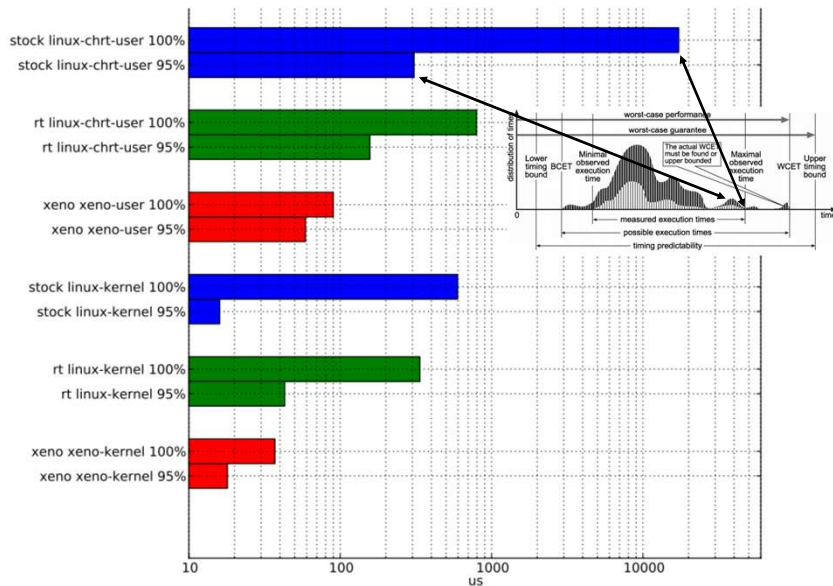
37

TTK4147 – 2025: Linux and Real-time Variants

NTNU

37

Execution time comparison between solutions



Be aware of the logarithmic scale!

38

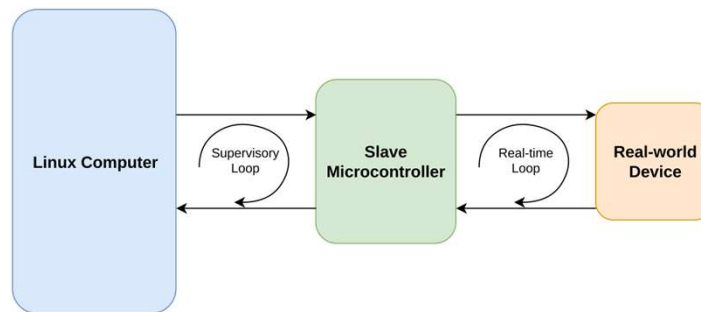
TTK4147 – 2025: Linux and Real-time Variants

NTNU

38

Strategy 3: Slave Microcontroller

- Outsourcing some of the real-time tasks to a specialized microcontroller.
 - The Linux computer act as a supervisor.
- Suitable (and usual) for systems that have a small, separate real-time component.



39

TTK4147 – 2025: Linux and Real-time Variants

NTNU

39

Embedded Linux

- Advantages using Linux:
 - Customizable and configurable.
 - Runs on a large number of architectures.
 - Huge, active user community.
 - Free.
- Disadvantages:
 - Not hard real-time by default.
 - There are too many alternatives. ?? Some are tailormade for the HW (Raspbian)
 - Versions ??
 - Software packages ??
 - Difficult ??
 - Could be less expensive to buy a working system than to spend a long time configuring a free Linux system. ??
 - Hardware capability issues (but better than many OSes).

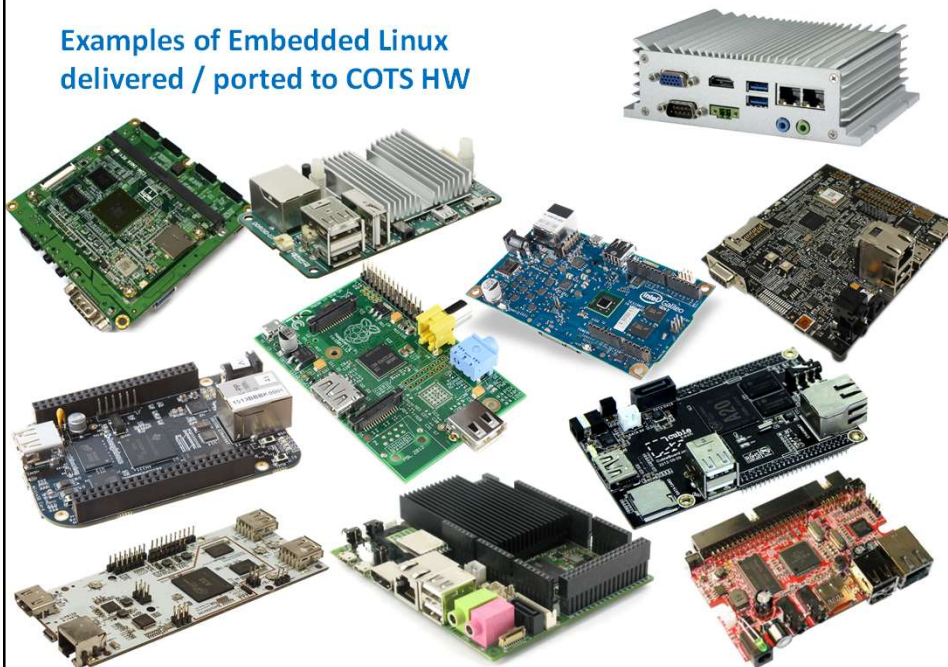
40

TTK4147 – 2025: Linux and Real-time Variants

NTNU

40

Examples of Embedded Linux delivered / ported to COTS HW



41

TTK4147 – 2025: Linux and Real-time Variants



41

Android

- An embedded environment that runs on Linux.
- Will not be covered in TTK4147.



42

TTK4147 – 2025: Linux and Real-time Variants



42

Raspberry Pi

- Both used as a General Purpose System or as an Embedded System
- Should use an OS depending on the purpose of the system:
 - Raspberry Pi OS (Raspbian), based on Debian
 - Ubuntu
 - PREEMPT_RT, patched version of Raspberry Pi OS
- (Used in lab in TTK4147.)
- Applicable for development of SW



43

TTK4147 – 2025: Linux and Real-time Variants



43

What about C/C++?

- Real-time programs are *almost* always developed with C/C++.
 - Direct interaction with hardware.
 - No virtual machine run-time (like Python/Java).

44

TTK4147 – 2025: Linux and Real-time Variants



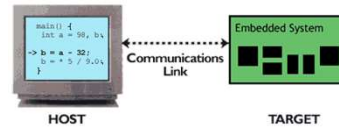
44

How to develop C/C++ programs for embedded systems?

- Use the host-target model(?)
- Where (on host or target) should the different tasks be performed?
 - Writing the code
 - Building the code
 - Debugging
- Embedded Development:

Alternatives, in general:

 - Use only Target (for instance Raspberry Pi)
 - Develop on Host, Built on Target
 - Develop and Built on Host, Transfer to Target
 - Cross-compile
 - Cross-compile and Flash
 - Embedded Linux Build Systems



45

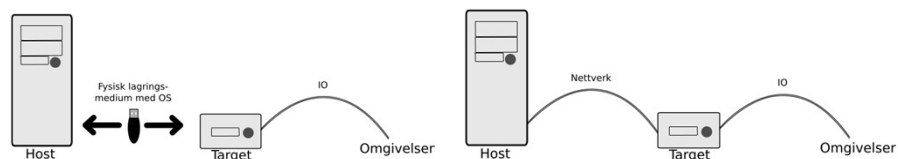
TTK4147 – 2025: Linux and Real-time Variants



45

Embedded Development

- In embedded development there is usually one or more targets and one host computer involved.
 - **Host computer** is the general purpose computer you use to program and compile the program on.
 - **Target computer** is the embedded computer that runs the developed code.
- We often compile on the host, because it is normally a more powerful computer.
 - Will often require cross-compiling.
 - Must move compiled software from host to target



46

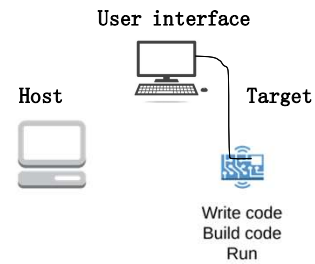
TTK4147 – 2025: Linux and Real-time Variants



46

Alternatively, Only use Target

- Do everything on the hardware the system will run on.
 - Often less powerful hardware than a desktop or laptop.
 - Often limited development tools.
 - But easy to set up
- It takes a Beaglebone Black approximately 12 hours to build its own Linux Kernel
- Not exactly the best development platform... (but getting better and better, often a question of cost of target.)



47

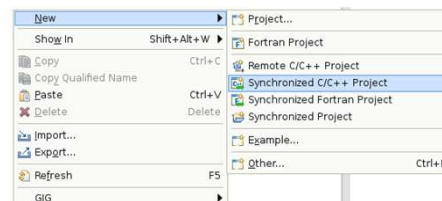
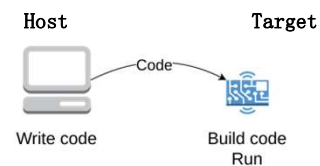
TTK4147 – 2025: Linux and Real-time Variants



47

Develop on Host, Build on Target

- A desktop or laptop will typically have better development tools.
 - Develop code on host.
 - Transfer code to target.
 - Build and run on target.
- Still have the problem that target computer might be slow.
 - Might even not have the necessary build tools.
- Eclipse (IDE) functionality for this type of setup, which is called "Synchronized C/C++ project".
 - Available as a downloadable plugin.
 - Syncs code on host and target.
 - Build on target.



48

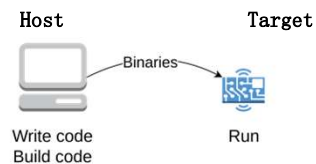
TTK4147 – 2025: Linux and Real-time Variants



48

Develop and Build on Host, Transfer to Target

- Develop and build a program as if should run on host.
 - Transfer binaries built on host to target.
 - Program should work the same way on target as it did on host.
- Will NOT work if host and targets are too different.
 - The same binaries will not work on different CPU platforms.
 - The same binaries will not work if host and targets have different shared libraries.
- Will be difficult to develop and test functionality that only the target has.
 - Connection with other parts of a larger system.
 - I/O devices.



49

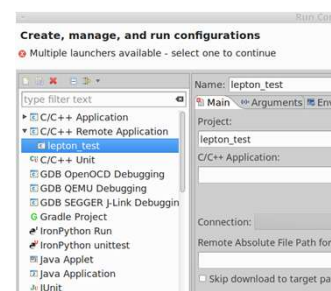
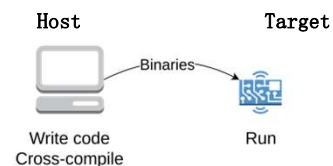
TTK4147 – 2025: Linux and Real-time Variants



49

Cross-compile

- A cross-compiler is a compiler that runs on one computer (host) and make binaries for another (target).
- Very useful for embedded development.
- Challenging to set up, but is usually the best solution when it works.
 - The host need a specialized tool (the cross compiler).
 - The host need a copy of the targets libraries.
 - [Need some method of transferring the binary and run it remotely.](#)
- Eclipse has support for this as well, called "C/C++ Remote Application".
 - Have to set up cross compilation.
 - Special "run configuration" that will transfer binary to target and run it.
 - Can also be used for debugging.



50

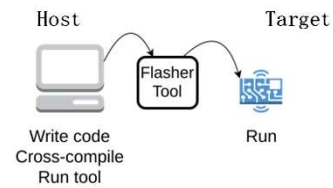
TTK4147 – 2025: Linux and Real-time Variants



50

Cross-compile and Flash

- For hardware that does not have removable storage, network etc.
 - Use an external tool or debugger, often to *flash* some content to some internal flash memory.
 - The tool can be external (JTAG used for Atmel micro-controllers)
 - Or on board (Bootloader on Arduino lets you upload programs without external device).
- Examples of this:
 - Different types of Atmel micro-controllers. (Atmel Studio and use of ICE as in lab exercise)
 - ARM cortex-m0 and m3 micro-controllers.



51

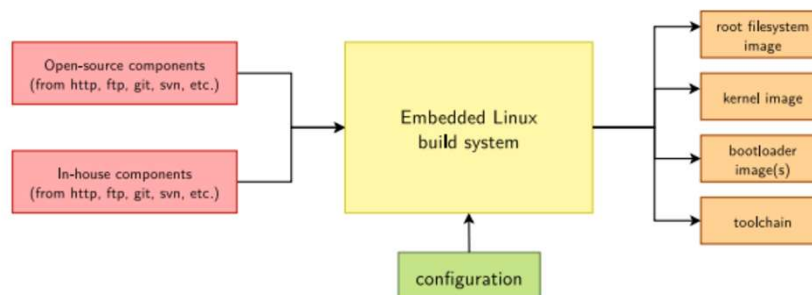
TTK4147 – 2025: Linux and Real-time Variants



51

Embedded Linux Build Systems

- Compromise between using a *binary distribution* (ubuntu, debian, etc) and having to make everything from scratch.
 - Almost full control over what is included in your system.
 - Built from source.
 - But still relatively easy to make.
- Based on configuration files and recipes.
 - Easy to reproduce a specific solution.



52

TTK4147 – 2025: Linux and Real-time Variants



52

Busybox

- All Linux systems (and other UNIX-like systems) need a set of basic applications (that ease the user's work), typically found in GNU coreutils, such as:
 - ls - list files
 - cp - copy files
 - ifconfig - manage network devices
 - grep - search for file content
 - etc.
- **Busybox - The Swiss Army Knife of Embedded Linux.**
 - Implements a collection of the common tools for embedded Linux.
 - Configurable, as uClibc, so you only need to build the tools you need.
 - One single binary file
 - All tools are within the same executable, binary file /bin/busybox.
 - E.g. you execute «ls» by running «/bin/busybox ls».



55

TTK4147 – 2025: Linux and Real-time Variants



55

Embedded Linux vs QNX/VxWorks

- This lecture has given a brief overview of the different ways to set up an embedded Linux system and development environment.
- All of this is provided to you if you use a commercial system as QNX or VxWorks.
 - You will get a cross-toolchain that build QNX/VxWorks programs on your desktop computer.
 - You will get tools to configure and make a file system or image file.
- The point is, the *freeness* of Linux comes at a cost.
 - Need to make a compromise between cost of buying a system and cost of setting up a system.



56

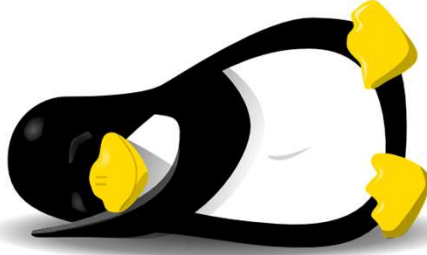
TTK4147 – 2025: Linux and Real-time Variants



56

Also use

Blackboard → TTK4147 Sanntidssystemer (2025 HØST) →
Learning materials → Questions



57 TTK4147 – 2025: Linux and Real-time Variants NTNU

57

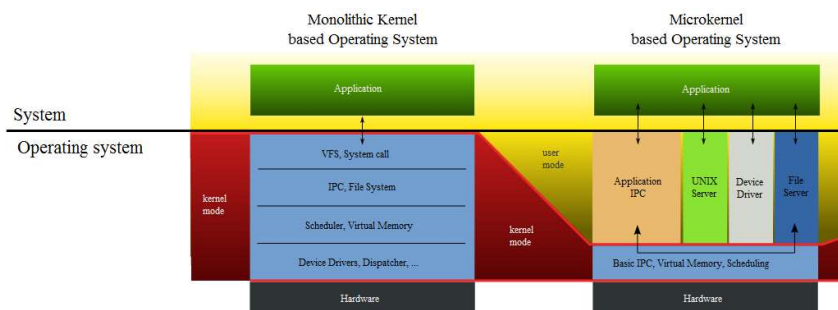
Endings

2022-15

What is the main difference between the monolithic kernel model and the microkernel model? Give your answer in one short paragraph.

Monolithic kernel: the whole operating system runs in kernel mode.

Microkernel: most operating system tasks operate outside kernel mode, the kernel deals with message passing, interrupts etc.



58

TTK4147 – 2025: Linux and Real-time Variants

NTNU

58

2021-14

Linux is not ideal for real-time systems. Mention strategies that can be used to make it more real-time capable.

1. Make the Linux kernel fully preemptive, with `PREEMPT_RT`, thus more predictable.
 - Effect may be uncertain. Tuning the SW
2. Add a small real-time co-kernel that runs «below» Linux.
 - RTAI (Real-time application interface) SW addition
 - Xenomai (Cobalt core)
3. Outsource the hard real-time tasks to a slave microcontroller
 - A simple microcontroller without OS like AVR provide better control of latencies. HW addition
 - Interface between master CPU and slave microcontroller with UART, SPI, I2C etc.

59

TTK4147 – 2025: Linux and Real-time Variants



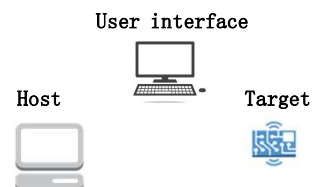
59

2020-1f

How can one develop a real-time system for a small programmable microcontroller with no user interface? The question asks for high level concepts, how and where should the system be developed?.

Alternatives:

- ~~Use only Target (for instance Raspberry Pi)~~
- Develop on Host, Built on Target
- Develop and Built on Host, Transfer to Target
- Cross-compile
- Cross-compile and Flash



60

TTK4147 – 2025: Linux and Real-time Variants



60